arXiv:1804.08328v2 [cs.CV] 20 Oct 2026

Taskonomy: Disentangling Task Transfer Learning

Amir R. Zamir[1,2] Alexander Sax[1*] William Shen[1*] Leonidas Guibas[1] Jitendra Malik[2] Silvio Savarese[1]

[1] Stanford University [2] University of California, Berkeley

http://taskonomy.stanford.edu/

Abstract

*Do visual tasks have a relationship, or are they unrelated? For instance, could having surface normals simplify estimating the depth of an image? Intuition answers these questions positively, implying existence of a **structure** among visual tasks. Knowing this structure has notable values; it is the concept underlying transfer learning and provides a principled way for identifying redundancies across tasks, in order to, for instance, seamlessly reuse supervision among related tasks or solve many tasks in one system without piling up the complexity.*

We propose a fully computational approach for modeling the structure of the space of visual tasks. This is done via finding (first and higher-order) transfer learning dependencies across a dictionary of twenty-six 2D, 2.5D, 3D, and semantic tasks in a latent space. The product is a computational taxonomic map for task transfer learning. We study the consequences of this structure, e.g. nontrivial emerged relationships, and exploit them to reduce the demand for labeled data. For example, we show that the total number of labeled datapoints needed for solving a set of 10 tasks can be reduced by roughly $\frac{2}{3}$ (compared to training independently) while keeping the performance nearly the same. We provide a set of tools for computing and probing this taxonomical structure including a solver that users can employ to devise efficient supervision policies for their use cases.

1. Introduction

Object recognition, depth estimation, edge detection, pose estimation, etc are examples of common vision tasks deemed useful and tackled by the research community. Some of them have rather clear relationships: we understand that surface normals and depth are related (one is a derivate of the other), or vanishing points in a room are useful for orientation and layout estimation. Other relationships are less clear: how edge detection and the shading in a room can, together, perform pose estimation.

*Equal.

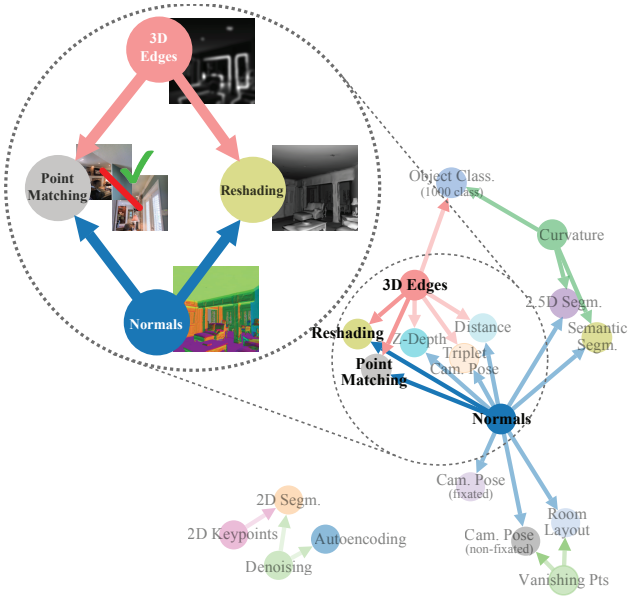


Figure 1: **A sample task structure discovered by the computational task taxonomy (*taskonomy*)**. It found that, for instance, by combining the learned features of a surface normal estimator and occlusion edge detector, good networks for reshading and point matching can be rapidly trained with little labeled data.

The field of computer vision has indeed gone far without explicitly using these relationships. We have made remarkable progress by developing advanced learning machinery (e.g. ConvNets) capable of finding complex mappings from X to Y when many pairs of (x, y) s.t. $x \in X, y \in Y$ are given as training data. This is usually referred to as fully supervised learning and often leads to problems being solved in isolation. Siloing tasks makes training a new task or a comprehensive perception system a Sisyphean challenge, whereby each task needs to be learned individually from scratch. Doing so ignores their quantifiably useful relationships leading to a massive labeled data requirement.

Alternatively, a model aware of the relationships among tasks demands less supervision, uses less computation, and behaves in more predictable ways. Incorporating such a structure is the first stepping stone towards develop-

ing provably efficient comprehensive/universal perception models [34, 4], i.e. ones that can solve a large set of tasks before becoming intractable in supervision or computation demands. However, this task space structure and its effects are still largely unknown. The relationships are non-trivial, and finding them is complicated by the fact that we have imperfect learning models and optimizers. In this paper, we attempt to shed light on this underlying structure and present a framework for mapping the space of visual tasks. Here what we mean by "structure" is a collection of computationally found relations specifying which tasks supply useful information to another, and by how much (see Fig. 1).

We employ a fully computational approach for this purpose, with neural networks as the adopted computational function class. In a feedforward network, each layer successively forms more abstract representations of the input containing the information needed for mapping the input to the output. These representations, however, can transmit statistics useful for solving other outputs (tasks), presumably if the tasks are related in some form [83, 19, 58, 46]. This is the basis of our approach: we compute an affinity matrix among tasks based on whether the solution for one task can be sufficiently easily read out of the representation trained for another task. Such transfers are exhaustively sampled, and a Binary Integer Programming formulation extracts a globally efficient transfer policy from them. We show this model leads to solving tasks with far less data than learning them independently and the resulting structure holds on common datasets (ImageNet [78] and Places [104]).

Being fully computational and representation-based, the proposed approach avoids imposing prior (possibly incorrect) assumptions on the task space. This is crucial because the priors about task relations are often derived from either human intuition or analytical knowledge, while neural networks need not operate on the same principles [63, 33, 40, 45, 102, 88]. For instance, although we might expect depth to transfer to surface normals better (derivatives are easy), the opposite is found to be the better direction in a computational framework (i.e. suited neural networks better).

An interactive taxonomy solver which uses our model to suggest data-efficient curricula, a live demo, dataset, and code are available at http://taskonomy.stanford.edu/.

2. Related Work

Assertions of existence of a structure among tasks date back to the early years of modern computer science, e.g. with Turing arguing for using learning elements [95, 98] rather than the final outcome or Jean Piaget's works on developmental stages using previously learned stages as sources [74, 39, 38], and have extended to recent works [76, 73, 50, 18, 97, 61, 11, 66]. Here we make an attempt to actually find this structure. We acknowledge that this is related to a breadth of topics, e.g. compositional modeling [35, 10, 13, 23, 55, 92, 90], homomorphic cryptography [42], lifelong learning [93, 15, 85, 84], functional maps [71], certain aspects of Bayesian inference and Dirichlet processes [54, 91, 90, 89, 37, 39], few-shot learning [81, 25, 24, 70, 86], transfer learning [75, 84, 29, 64, 67, 59], un/semi/self-supervised learning [22, 8, 17, 103, 19, 83], which are studied across various fields [73, 94, 12]. We review the topics most pertinent to vision within the constraints of space:

Self-supervised learning methods leverage the inherent relationships between tasks to learn a desired expensive one (e.g. object detection) via a cheap surrogate (e.g. colorization) [68, 72, 17, 103, 100, 69]. Specifically, they use a manually-entered local part of the structure in the task space (as the surrogate task is manually defined). In contrast, our approach models this large space of tasks in a computational manner and can discover obscure relationships.

Unsupervised learning is concerned with the redundancies in the input domain and leveraging them for forming compact representations, which are usually agnostic to the downstream task [8, 49, 20, 9, 32, 77]. Our approach is not unsupervised by definition as it is not agnostic to the tasks. Instead, it models the space tasks belong to and in a way utilizes the *functional* redundancies among tasks.

Meta-learning generally seeks performing the learning at a level higher than where conventional learning occurs, e.g. as employed in reinforcement learning [21, 31, 28], optimization [2, 82, 48], or certain architectural mechanisms [27, 30, 87, 65]. The motivation behind meta learning has similarities to ours and our outcome can be seen as a computational meta-structure of the space of tasks.

Multi-task learning targets developing systems that can provide multiple outputs for an input in one run [50, 18]. Multi-task learning has experienced recent progress and the reported advantages are another support for existence of a useful structure among tasks [93, 100, 50, 76, 73, 50, 18, 97, 61, 11, 66]. Unlike multi-task learning, we explicitly model the relations among tasks and extract a meta-structure. The large number of tasks we consider also makes developing one multi-task network for all infeasible.

Domain adaption seeks to render a function that is developed on a certain domain applicable to another [44, 99, 5, 80, 52, 26, 36]. It often addresses a shift in the *input* domain, e.g. webcam images to D-SLR [47], while the task is kept the same. In contrast, our framework is concerned with *output* (task) space, hence can be viewed as *task/output adaptation*. We also perform the adaptation in a larger space among many elements, rather than two or a few.

In the context of our approach to modeling transfer learning across tasks:

Learning Theoretic approaches may overlap with any of the above topics and usually focus on providing generalization guarantees. They vary in their approach: e.g. by modeling transferability with the transfer family required

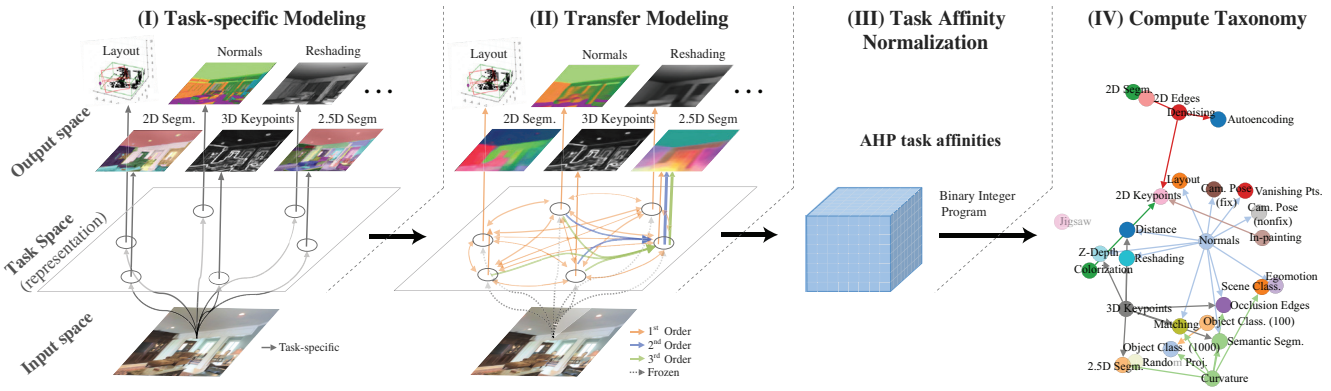


Figure 2: **Computational modeling of task relations and creating the taxonomy.** From left to right: I. Train task-specific networks. II. Train (first order and higher) transfer functions among tasks in a latent space. III. Get normalized transfer affinities using AHP (Analytic Hierarchy Process). IV. Find global transfer taxonomy using BIP (Binary Integer Program).

to map a hypothesis for one task onto a hypothesis for another [7], through information-based approaches [60], or through modeling inductive bias [6]. For these guarantees, learning theoretic approaches usually rely on intractable computations, or avoid such computations by restricting the model or task. Our method draws inspiration from theoretical approaches but eschews (for now) theoretical guarantees in order to use modern neural machinery.

3. Method

We define the problem as follows: we want to maximize the collective performance on a set of tasks $\mathcal{T} = \{t_1, ..., t_n\}$, subject to the constraint that we have a limited supervision budget γ (due to financial, computational, or time constraints). We define our supervision budget γ to be the maximum allowable number of tasks that we are willing to train from scratch (i.e. *source* tasks). The task dictionary is defined as $\mathcal{V}$=$\mathcal{T} \cup \mathcal{S}$ where $\mathcal{T}$ is the set of tasks which we want solved (*target*), and $\mathcal{S}$ is the set of tasks that can be trained (*source*). Therefore, $\mathcal{T} - \mathcal{T} \cap \mathcal{S}$ are the tasks that we want solved but cannot train ("target-only"), $\mathcal{T} \cap \mathcal{S}$ are the tasks that we want solved but could play as source too, and $\mathcal{S} - \mathcal{T} \cap \mathcal{S}$ are the "source-only" tasks which we may not directly care about to solve (e.g. jigsaw puzzle) but can be optionally used if they increase the performance on $\mathcal{T}$.

The **task** tax**onomy** (***taskonomy***) is a computationally found directed hypergraph that captures the notion of task transferability over any given task dictionary. An edge between a group of source tasks and a target task represents a feasible transfer case and its weight is the prediction of its performance. We use these edges to estimate the globally optimal transfer policy to solve $\mathcal{T}$. Taxonomy produces a family of such graphs, parameterized by the available supervision budget, chosen tasks, transfer orders, and transfer functions' expressiveness.

Taxonomy is built using a four step process depicted in Fig. 2. In stage I, a task-specific network for each task in $\mathcal{S}$ is trained. In stage II, all feasible transfers between sources and targets are trained. We include higher-order transfers which use multiple inputs task to transfer to one target. In stage III, the task affinities acquired from transfer function performances are normalized, and in stage IV, we synthesize a hypergraph which can predict the performance of any transfer policy and optimize for the optimal one.

A vision task is an abstraction read from a raw image. We denote a task t more formally as a function f_t which maps image I to $f_t(I)$. Our dataset, $\mathcal{D}$, contains for each task t a set of training pairs $(I, f_t(I))$, e.g. $(image, depth)$.

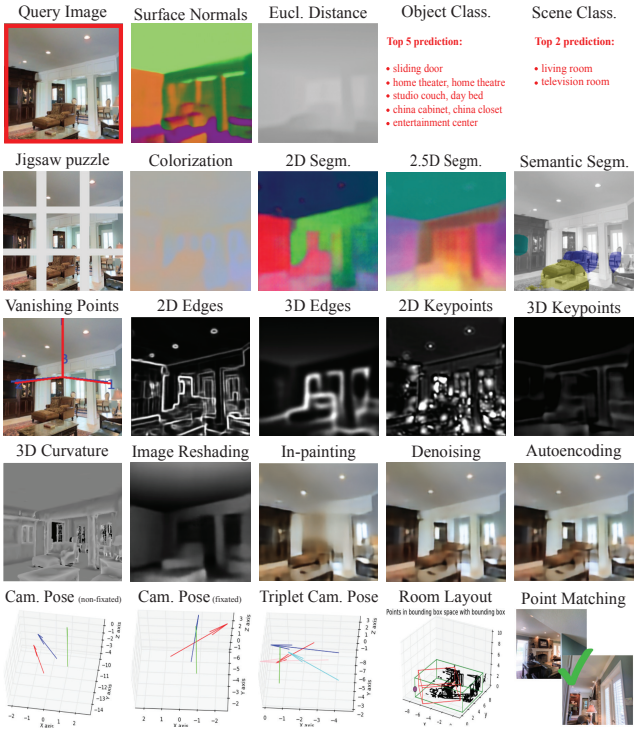


Figure 3: **Task Dictionary.** Outputs of 24 (of 26) task-specific networks for a query (top left). See results of applying frame-wise on a video here.

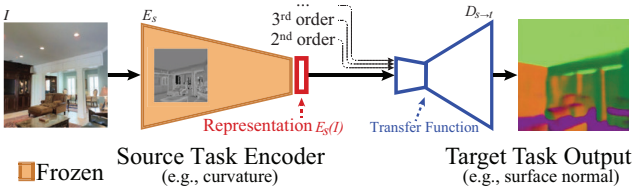


Figure 4: **Transfer Function**. A small readout function is trained to map representations of source task's frozen encoder to target task's labels. If order>1, transfer function receives representations from multiple sources.

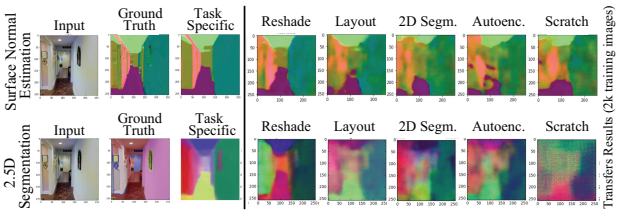


Figure 5: **Transfer results** to normals (upper) and 2.5D Segmentation (lower) from 5 different source tasks. The spread in transferability among different sources is apparent, with reshading among top-performing ones in this case. Task-specific networks were trained on 60x more data. "Scratch" was trained from scratch without transfer learning.

Task Dictionary: Our mapping of task space is done via (26) tasks included in the dictionary, so we ensure they cover common themes in computer vision (2D, 3D, semantics, etc) to elucidate fine-grained structures of task space. See Fig. 3 for some of the tasks with detailed definition of each task provided in the supplementary material. We include tasks with various levels of abstraction, ranging from solvable by a simple kernel convolved over the image (e.g. edge detection) to tasks requiring basic understanding of scene geometry (e.g. vanishing points) and more abstract ones involving semantics (e.g. scene classification).

It is critical to note the task dictionary is meant to be a *sampled set, not an exhaustive list*, from a denser space of all conceivable visual tasks/abstractions. Sampling gives us a tractable way to sparsely model a dense space, and the hypothesis is that (subject to a proper sampling) the derived model should generalize to out-of-dictionary tasks. The more regular / better sampled the space, the better the generalization. We evaluate this in Sec. 4.2 with supportive results. For evaluation of the robustness of results w.r.t the choice of dictionary, see the supplementary material.

Dataset: We need a dataset that has annotations for *every task on every image*. Training all of our tasks on exactly the same pixels eliminates the possibility that the observed transferabilities are affected by different input data peculiarities rather than only task intrinsics. There has not been such a dataset of scale made of real images, so we created a dataset of 4 million images of indoor scenes from about 600 buildings; every image has an annotation for every task. The images are registered on and aligned with building-wide meshes similar to [3, 101, 14] enabling us to programmatically compute the ground truth for many tasks without human labeling. For the tasks that still require labels (e.g. scene classes), we generate them using Knowledge Distillation [43] from known methods [104, 57, 56, 78]. See the supplementary material for full details of the process and a user study on the final quality of labels generated using Knowledge Distillation (showing $< 7\%$ error).

3.1. Step I: Task-Specific Modeling

We train a fully supervised task-specific network for each task in $\mathcal{S}$. Task-specific networks have an encoder-decoder architecture homogeneous across all tasks, where the encoder is large enough to extract powerful representations, and the decoder is large enough to achieve a good performance but is much smaller than the encoder.

3.2. Step II: Transfer Modeling

Given a source task s and a target task t, where $s \in \mathcal{S}$ and $t \in \mathcal{T}$, a transfer network learns a small readout function for t given a statistic computed for s (see Fig 4). The statistic is the representation for image I from the encoder of s: $E_s(I)$. The readout function ($D_{s\to t}$) is parameterized by $\theta_{s\to t}$ minimizing the loss L_t:

$$D_{s\to t} := \arg\min_{\theta} \mathbb{E}_{I\in\mathcal{D}}\Big[L_t\Big(D_\theta\big(E_s(I)\big), f_t(I)\Big)\Big], \quad (1)$$

where $f_t(I)$ is ground truth of t for image I. $E_s(I)$ may or may not be sufficient for solving t depending on the relation between t and s (examples in Fig. 5). Thus, the performance of $D_{s\to t}$ is a useful metric as task affinity. We train transfer functions for all feasible source-target combinations.

Accessibility: For a transfer to be successful, the latent representation of the source should both be *inclusive* of sufficient information for solving the target and have the information *accessible*, i.e. easily extractable (otherwise, the raw image or its compression based representations would be optimal). Thus, it is crucial for us to adopt a low-capacity (small) architecture as transfer function trained with a small amount of data, in order to measure transferability conditioned on being highly accessible. We use a shallow fully convolutional network and train it with little data (8x to 120x less than task-specific networks).

Higher-Order Transfers: Multiple source tasks can contain complementary information for solving a target task (see examples in Fig 6). We include higher-order transfers which are the same as first order but receive multiple representations in the input. Thus, our transfers are functions $D : \wp(\mathcal{S}) \to \mathcal{T}$, where $\wp$ is the powerset operator.

As there is a combinatorial explosion in the number of feasible higher-order transfers ($|\mathcal{T}| \times \binom{|\mathcal{S}|}{k}$ for k^{th} order), we employ a sampling procedure with the goal of filtering out higher-order transfers that are less likely to yield good results, without training them. We use a beam search: for

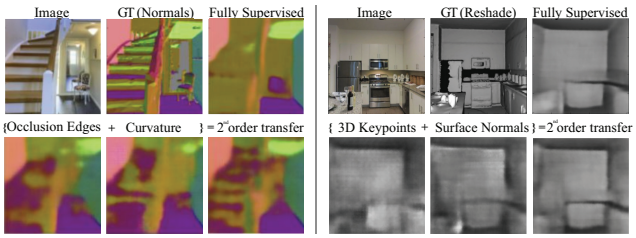


Figure 6: **Higher-Order Transfers.** Representations can contain complementary information. E.g. by transferring simultaneously from 3D Edges and Curvature individual stairs were brought out. See our publicly available interactive transfer visualization page for more examples.

transfers of order $k \leq 5$ to a target, we select its 5 best sources (according to 1st order performances) and include all of their order-k combination. For $k \geq 5$, we use a beam of size 1 and compute the transfer from the top k sources.

Transitive Transfers: We examined if transitive task transfers ($s \rightarrow t_1 \rightarrow t_2$) could improve the performance over their direct counterpart ($s \rightarrow t_2$), but found that the two had equal performance in almost all cases in both high-data and low-data scenarios. The experiment is provided in the supplementary material. Therefore, we need not consider the cases where branching would be more than one level deep when searching for the optimal transfer path.

3.3. Step III: Ordinal Normalization using Analytic Hierarchy Process (AHP)

We want to have an affinity matrix of transferabilities across tasks. Aggregating the raw losses/evaluations $L_{s \rightarrow t}$ from transfer functions into a matrix is obviously problematic as they have vastly different scales and live in different spaces (see Fig. 7-left). Hence, a proper normalization is needed. A naive solution would be to linearly rescale each row of the matrix to the range $[0, 1]$. This approach fails when the actual output quality increases at different speeds w.r.t. the loss. As the loss-quality curve is generally unknown, such approaches to normalization are ineffective.

Instead, we use an *ordinal* approach in which the output quality and loss are only assumed to change monotonically. For each t, we construct W_t a pairwise tournament matrix between all feasible sources for transferring to t. The element at (i, j) is the percentage of images in a held-out test set, $\mathcal{D}_{test}$, on which s_i transfered to t better than s_j did (i.e. $D_{s_i \rightarrow t}(I) > D_{s_j \rightarrow t}(I)$).

We clip this intermediate pairwise matrix W_t to be in $[0.001, 0.999]$ as a form of Laplace smoothing. Then we divide $W'_t = W_t / W_t^T$ so that the matrix shows how many times better s_i is compared to s_j. The final tournament ratio matrix is positive reciprocal with each element $w'_{i,j}$ of W'_t:

$$w'_{i,j} = \frac{\mathbb{E}_{I \in \mathcal{D}_{test}}[D_{s_i \rightarrow t}(I) > D_{s_j \rightarrow t}(I)]}{\mathbb{E}_{I \in \mathcal{D}_{test}}[D_{s_i \rightarrow t}(I) < D_{s_j \rightarrow t}(I)]}. \quad (2)$$

We quantify the final transferability of s_i to t as the corresponding (i^{th}) component of the principal eigenvector of W'_t (normalized to sum to 1). The elements of the principal eigenvector are a measure of centrality, and are proportional to the amount of time that an infinite-length random walk on W'_t will spend at any given source [62]. We stack the principal eigenvectors of W'_t for all $t \in \mathcal{T}$, to get an affinity matrix P ('p' for performance)—see Fig. 7, right.

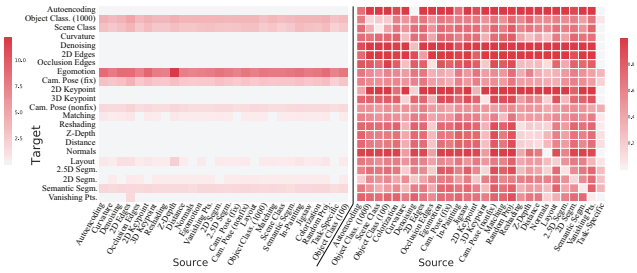


Figure 7: **First-order task affinity matrix** before (left) and after (right) Analytic Hierarchy Process (AHP) normalization. Lower means better transfered. For visualization, we use standard affinity-distance method $dist = e^{-\beta \cdot P}$ (where $\beta = 20$ and e is element-wise matrix exponential). See supplementary material for the full matrix with higher-order transfers.

This approach is derived from Analytic Hierarchy Process [79], a method widely used in operations research to create a total order based on multiple pairwise comparisons.

3.4. Step IV: Computing the Global Taxonomy

Given the normalized task affinity matrix, we need to devise a global transfer policy which maximizes collective performance across all tasks, while minimizing the used supervision. This problem can be formulated as subgraph selection where tasks are nodes and transfers are edges. The optimal subgraph picks the ideal source nodes and the best edges from these sources to targets while satisfying that the number of source nodes does not exceed the supervision budget. We solve this subgraph selection problem using Boolean Integer Programming (BIP), described below, which can be solved optimally and efficiently [41, 16].

Our transfers (edges), E, are indexed by i with the form $(\{s^i_1, \ldots, s^i_{m_i}\}, t^i)$ where $\{s^i_1, \ldots, s^i_{m_i}\} \subset \mathcal{S}$ and $t^i \in \mathcal{T}$. We define operators returning target and sources of an edge:

$$\left(\{s^i_1, \ldots, s^i_{m_i}\}, t^i\right) \xmapsto{sources} \{s^i_1, \ldots, s^i_{m_i}\}$$
$$\left(\{s^i_1, \ldots, s^i_{m_i}\}, t^i\right) \xmapsto{target} t^i.$$

Solving a task t by fully supervising it is denoted as $\left(\{t\}, t\right)$. We also index the targets $\mathcal{T}$ with j so that in this section, i represents edges and j represents targets.

The parameters of the problem are: the supervision budget (γ) and a measure of performance on a target from each of its transfers (p_i), i.e. the affinities from P. We can also optionally include additional parameters of: r_j specifying the relative importance of each target task and ℓ_i specifying the relative cost of acquiring *labels* for each task.

The BIP is parameterized by a vector x where each transfer and each task is represented by a binary variable; x indicates which nodes are picked to be source and which transfers are selected. The canonical form for a BIP is:

$$\text{maximize: } c^T x \,,$$
$$\text{subject to: } Ax \preceq b \text{ and } x \in \{0,1\}^{|E|+|\mathcal{V}|} \,.$$

Each element c_i for a transfer is the product of the importance of its target task and its transfer performance:

$$c_i := r_{target(i)} \cdot p_i \,. \tag{3}$$

Hence, the *collective* performance on all targets is the summation of their individual AHP performance, p_i, weighted by the user specified importance, r_i.

Now we add three types of constraints via matrix A to enforce each feasible solution of the BIP instance corresponds to a valid subgraph for our transfer learning problem: *Constraint I:* if a transfer is included in the subgraph, all of its source nodes/tasks must be included too, *Constraint II:* each target task has exactly one transfer in, *Constraint III:* supervision budget is not exceeded.

Constraint I: For each row a_i in A we require $a_i \cdot x \leq b_i$, where

$$a_{i,k} := \begin{cases} |sources(i)| & \text{if } k = i \\ -1 & \text{if } (k - |E|) \in sources(i) \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

$$b_i = 0. \tag{5}$$

Constraint II: Via two rows $a_{|E|+j}$ and $a_{|E|+j+1}$, we enforce that target j has exactly one transfer:

$$\begin{aligned} a_{|E|+j,i} &:= \mathbb{1}_{\{target(i)=j\}}, & b_{|E|+j} &:= +1, \\ a_{|E|+j+1,i} &:= -\mathbb{1}_{\{target(i)=j\}}, & b_{|E|+j+1} &:= -1. \end{aligned} \tag{6}$$

Constraint III: the solution is enforced to not exceed the budget. Each transfer i is assigned a label cost ℓ_i, so

$$a_{|E|+2|\mathcal{V}|+1,i} := \ell_i, \qquad b_{|E|+2|\mathcal{V}|+1} := \gamma. \tag{7}$$

The elements of A not defined above are set to 0. The problem is now a valid BIP and can be optimally solved in a fraction of a second [41]. The BIP solution $\hat{x}$ corresponds to the optimal subgraph, which is our taxonomy.

4. Experiments

With 26 tasks in the dictionary (4 source-only tasks), our approach leads to training 26 fully supervised task-specific networks, 22×25 transfer networks in 1st order, and $22 \times \binom{25}{k}$ for k^{th} order, from which we sample according to the procedure in Sec. 3. The total number of transfer functions trained for the taxonomy was ∼3,000 which took 47,886 GPU hours on the cloud.

Out of 26 tasks, we use the following 4 as source-only tasks (described in Sec. 3) in the experiments: colorization, jigsaw puzzle, in-painting, random projection. However, the method is applicable to an arbitrary partitioning of the dictionary into $\mathcal{T}$ and $\mathcal{S}$. The interactive solver website allows the user to specify any desired partition.

Task	*avg*	*rand*	Task	*avg*	*rand*	Task	*avg*	*rand*
Denoising	100	99.9	Layout	99.6	89.1	Scene Class.	97.0	93.4
Autoenc.	100	99.8	2D Edges	100	99.9	Occ. Edges	100	95.4
Reshading	94.9	95.2	Pose (fix)	76.3	79.5	Pose (nonfix)	60.2	61.9
Inpainting	99.9	-	2D Segm.	97.7	95.7	2.5D Segm.	94.2	89.4
Curvature	78.7	93.4	Matching	86.8	84.6	Egomotion	67.5	72.3
Normals	99.4	99.5	Vanishing	99.5	96.4	2D Keypnt.	99.8	99.4
Z-Depth	92.3	91.1	Distance	92.4	92.1	3D Keypnt.	96.0	96.9
Mean	92.4	90.9						

Table 1: **Task-Specific Networks' Sanity:** Win rates vs. *random* (Gaussian) network representation readout and statistically informed guess *avg*.

Network Architectures: We preserved the architectural and training details across tasks as homogeneously as possible to avoid injecting any bias. The **encoder** architecture is identical across all task-specific networks and is a fully convolutional ResNet-50 without pooling. All **transfer** functions include identical shallow networks with 2 conv layers (concatenated channel-wise if higher-order). The loss (L_t) and **decoder**'s architecture, though, have to depend on the task as the output structures of different tasks vary; for all pixel-to-pixel tasks, e.g. normal estimation, the decoder is a 15-layer fully convolutional network; for low dimensional tasks, e.g. vanishing points, it consists of 2-3 FC layers. All networks are trained using the same hyperparameters regardless of task and on exactly the same input images. Tasks with more than one input, e.g. relative camera pose, share weights between the encoder towers. Transfer networks are all trained using the same hyperparameters as the task-specific networks, except that we anneal the learning rate earlier since they train much faster. Detailed definitions of architectures, training process, and experiments with different encoders can be found in the supplementary material.

Data Splits: Our dataset includes 4 million images. We made publicly available the models trained on full dataset, but for the experiments reported in the main paper, we used a subset of the dataset as the extracted structure stabilized and did not change when using more data (explained in Sec. 4.5). The used subset is partitioned into training (120k), validation (16k), and test (17k) images, each from non-overlapping sets of buildings. Our task-specific networks are trained on the training set and the transfer networks are trained on a subset of validation set, ranging from 1k images to 16k, in order to model the transfer patterns under different data regimes. In the main paper, we report all results under the 16k transfer supervision regime (∼10% of the split) and defer the additional sizes to the supplementary material and website (see Sec. 4.5). Transfer functions are evaluated on the test set.

How good are the trained task-specific networks? *Win rate (%)* is the proportion of test set images for which a baseline is beaten. Table 1 provides win rates of the task-specifc networks vs. two baselines. Visual outputs for a ran-

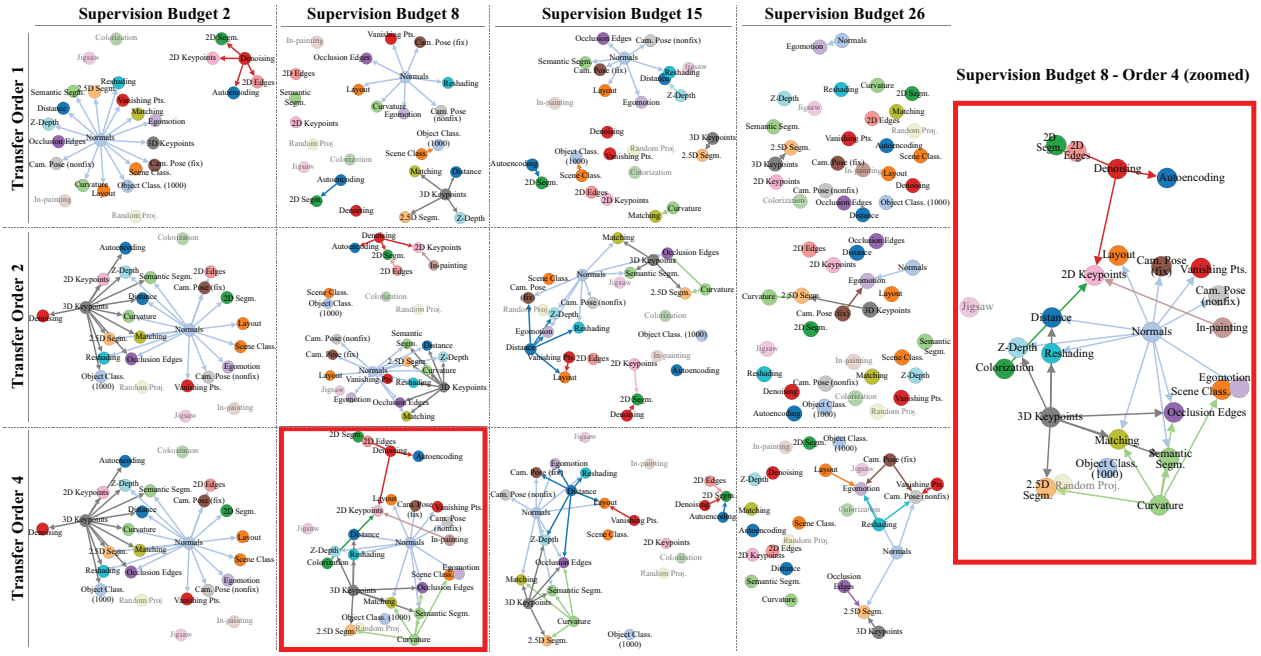


Figure 8: **Computed taxonomies** for solving 22 tasks given various supervision budgets (x-axes), and maximum allowed transfer orders (y-axes). One is magnified for better visibility. Nodes with incoming edges are target tasks, and the number of their incoming edges is the order of their chosen transfer function. Still transferring to some targets when tge budget is 26 (full budget) means certain transfers started performing better than their fully supervised task-specific counterpart. See the interactive solver website for color coding of the nodes by *Gain* and *Quality* metrics. Dimmed nodes are the source-only tasks, and thus, only participate in the taxonomy if found worthwhile by the BIP optimization to be one of the sources.

dom test sample are in Fig. 3. The high win rates in Table 1 and qualitative results show the networks are well trained and stable and can be relied upon for modeling the task space. See results of applying the networks on a YouTube video frame-by-frame here. A live demo for user uploaded queries is available here.

To get a sense of the quality of our networks vs. state-of-the-art task-specific methods, we compared our depth estimator vs. released models of [53] which led to outperforming [53] with a win rate of 88% and losses of 0.35 vs. 0.47 (further details in the supplementary material). In general, we found the task-specific networks to perform on par or better than state-of-the-art for many of the tasks, though we do not formally benchmark or claim this.

4.1. Evaluation of Computed Taxonomies

Fig. 8 shows the computed taxonomies optimized to solve the full dictionary, i.e. all tasks are placed in $\mathcal{T}$ and $\mathcal{S}$ (except for 4 source-only tasks that are in $\mathcal{S}$ only). This was done for various supervision budgets (columns) and maximum allowed order (rows) constraints. Still seeing transfers to some targets when the budget is 26 (full dictionary) means certain transfers became better than their fully supervised task-specific counterpart.

While Fig. 8 shows the structure and connectivity, Fig. 9 quantifies the results of taxonomy recommended transfer policies by two metrics of *Gain* and *Quality*, defined as:

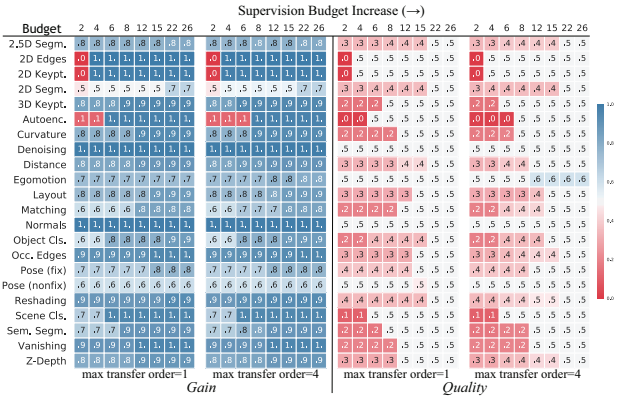
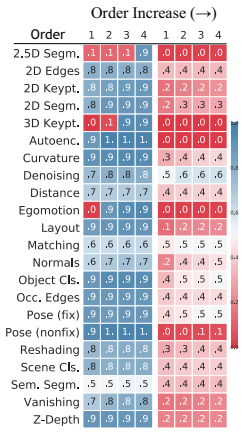


Figure 9: **Evaluation of taxonomy computed for solving the full task dictionary**. Gain (left) and Quality (right) values for each task using the policy suggested by the computed taxonomy, as the supervision budget increases(→). Shown for transfer orders 1 and 4.

Gain: win rate (%) against a network trained from scratch using the same training data as transfer networks'. That is, the best that could be done if transfer learning was not utilized. This quantifies the *gained* value by transferring.
Quality: win rate (%) against a fully supervised network trained with 120k images (gold standard).

Red (0) and Blue (1) represent outperforming the reference method on none and all of test set images, respectively (so the transition Red→White→Blue is desirable. White (.5) represents equal performance to reference).



Task	scratch	ImageNet[51]	Wang,[96]	Agrawal,[1]	Zamir,[100]	Zhang,[103]	Noroozi,[68]	full sup.	Taxonomy
Depth	88	88	93	89	88	84	86	43	-
	.03	.04	.04	.03	.04	.03	.03	**.02**	.02
Scene Cls.	80	52	83	74	74	71	75	15	-
	3.30	2.76	3.56	3.15	3.17	3.09	3.19	**2.23**	2.63
Sem. Segm.	78	79	82	85	76	78	84	21	-
	1.74	1.88	1.92	1.80	1.85	1.74	1.71	**1.42**	1.53
Object Cls.	79	54	82	76	75	76	76	34	-
	4.08	3.57	4.27	3.99	3.98	4.00	3.97	**3.26**	3.46
Normals	97	98	98	98	98	97	97	6	-
	.22	.30	.34	.28	.28	.23	.24	**.12**	.15
2.5D Segm.	80	93	92	89	90	84	87	40	-
	.21	.34	.34	.26	.29	.22	.24	**.16**	.17
Occ. Edges	93	96	95	93	94	93	94	42	-
	.16	.19	.18	.17	.18	.16	.17	**.12**	.13
Curvature	88	94	89	85	88	92	88	29	-
	.25	.28	.26	.25	.26	.26	.25	**.21**	.22
Egomotion	79	78	83	77	76	74	71	59	-
	8.60	8.58	9.26	8.41	8.34	8.15	7.94	7.32	**6.85**
Layout	80	76	85	79	77	78	70	36	-
	.66	.66	.85	.65	.65	.62	.54	**.37**	.41

Figure 10: **Generalization to Novel Tasks**. Each row shows a novel test task. Left: Gain and Quality values using the devised "all-for-one" transfer policies for novel tasks for orders 1-4. Right: Win rates (%) of the transfer policy over various self-supervised methods, ImageNet features, and scratch are shown in the colored rows. Note the large margin of win by taxonomy. The uncolored rows show corresponding loss values.

Each column in Fig. 9 shows a supervision budget. As apparent, good results can be achieved even when the supervision budget is notably smaller than the number of solved tasks, and as the budget increases, results improve (expected). Results are shown for 2 maximum allowed orders.

4.2. Generalization to Novel Tasks

The taxonomies in Sec. 4.1 were optimized for solving all tasks in the dictionary. In many situations, a practitioner is interested in a single task which even may not be in the dictionary. Here we evaluate how taxonomy transfers to a novel out-of-dictionary task with little data.

This is done in an all-for-one scenario where we put one task in $\mathcal{T}$ and all others in $\mathcal{S}$. The task in $\mathcal{T}$ is target-only and has no task-specific network. Its limited data (16k) is used to train small transfer networks to sources. This basically *localizes* where the target would be in the taxonomy.

Fig. 10 (left) shows the *Gain* and *Quality* of the transfer policy found by the BIP for each task. Fig. 10 (right) compares the taxonomy suggested policy against some of the best existing self-supervised methods [96, 103, 68, 100, 1], ImageNet FC7 features [51], training from scratch, and a fully supervised network (gold standard).

The results in Fig. 10 (right) are noteworthy. The large win margin for taxonomy shows that carefully selecting transfer policies depending on the target is superior to fixed transfers, such as the ones employed by self-supervised methods. ImageNet features which are the most popular off-the-shelf features in vision are also outperformed by those policies. Additionally, though the taxonomy transfer policies lose to fully supervised networks (gold standard) in most cases, the results often get close with win rates in 40% range. These observations suggests the space has a rather

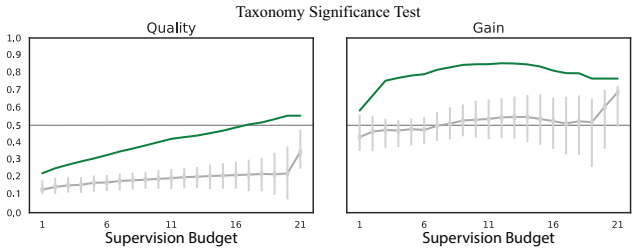


Figure 11: **Structure Significance.** Our taxonomy compared with random transfer policies (random feasible taxonomies that use the maximum allowable supervision budget). Y-axis shows *Quality* or *Gain*, and X-axis is the supervision budget. Green and gray represent our taxonomy and random connectivities, respectively. Error bars denote 5th–95th percentiles.

predicable and strong structure. For graph visualization of the all-for-one taxonomy policies please see the supplementary material. The solver website allows generating the taxonomy for arbitrary sets of target-only tasks.

4.3. Significance Test of the Structure

The previous evaluations showed good transfer results in terms of *Quality* and *Gain*, but how crucial is it to use our taxonomy to choose smart transfers over just choosing *any* transfer? In other words, how *significant/strong* is the discovered structure of task space? Fig. 11 quantifies this by showing the performance of our taxonomy versus a large set of taxonomies with random connectivities. Our taxonomy outperformed all other connectivities by a large margin signifying both existence of a strong structure in the space as well as a good modeling of it by our approach. Complete experimental details is available in supplementary material.

4.4. Evaluation on MIT Places & ImageNet

To what extent are our findings dataset dependent, and would the taxonomy change if done on another dataset? We examined this by finding the ranking of all tasks for transferring to two target tasks of object classification and scene classification on our dataset. We then fine tuned our task-specific networks on other datasets (MIT Places [104] for scene classification, ImageNet [78] for object classification) and evaluated them on their respective test sets and metrics. Fig. 12 shows how the results correlate with taxonomy's ranking from our dataset. The Spearman's rho between the taxonomy ranking and the Top-1 ranking is 0.857 on Places and 0.823 on ImageNet showing a notable correlation. See supplementary material for complete experimental details.

4.5. Universality of the Structure

We employed a computational approach with various design choices. It is important to investigate how specific to those the discovered structure is. We did stability tests by computing the variance in our output when making changes in one of the following system choices: **I. architecture of**

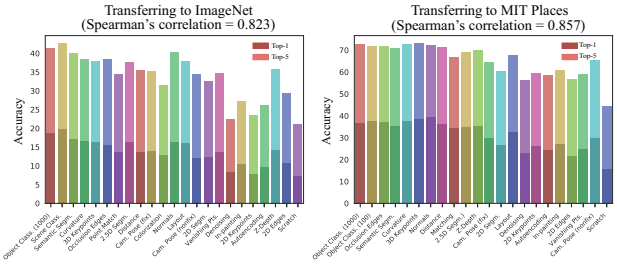


Figure 12: **Evaluating the discovered structure on other datasets: ImageNet [78] (left) for object classification and MIT Places [104] (right) for scene classification.** Y-axis shows accuracy on the external benchmark while bars on x-axis are ordered by taxonomy's predicted performance based on our dataset. A monotonically decreasing plot corresponds to preserving identical orders and perfect generalization.

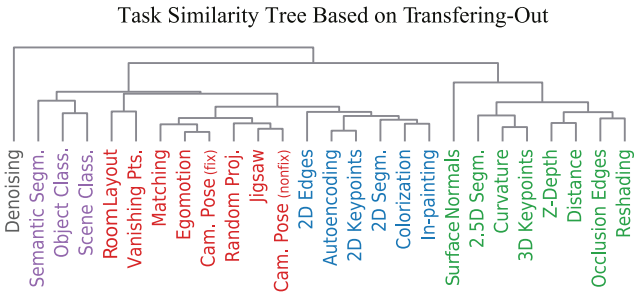


Figure 13: **Task Similarity Tree.** Agglomerative clustering of tasks based on their transferring-out patterns (i.e. using columns of normalized affinity matrix as task features). 3D, 2D, low dimensional geometric, and semantic tasks clustered together using a fully computational approach.

task-specific networks, II. architecture of transfer function networks, III. amount of data available for training transfer networks, IV. datasets, V. data splits, VI. choice of dictionary. Overall, despite injecting large changes (e.g. varying the size of training data of transfer functions by 16x, size and architecture of task-specific networks and transfer networks by 4x), we found the outputs to be remarkably stable leading to almost no change in the output taxonomy computed on top. Detailed results and experimental setup of each tests are reported in the supplementary material.

4.6. Task Similarity Tree

Thus far we showed the task space has a structure, measured this structure, and presented its utility for transfer learning via devising transfer policies. This structure can be presented in other manners as well, e.g. via a metric of similarity across tasks. Figure 13 shows a similarity tree for the tasks in our dictionary. This is acquired from agglomerative clustering of the tasks based on their transferring-out behavior, i.e. using columns of normalized affinity matrix P as feature vectors for tasks. The tree shows how tasks would be hierarchically positioned w.r.t. to each other when measured based on providing information for solving other tasks; the closer two tasks, the more similar their role in transferring to other tasks. Notice that the 3D, 2D, low dimensional geometric, and semantic tasks are found to cluster together using a fully computational approach, which matches the intuitive expectations from the structure of task space. The transfer taxonomies devised by BIP are consistent with this tree as BIP picks the sources in a way that all of these modes are quantitatively best covered, subject to the given budget and desired target set.

5. Limitations and Discussion

We presented a method for modeling the space of visual tasks by way of transfer learning and showed its utility in reducing the need for supervision. The space of tasks is an interesting object of study in its own right and we have only scratched the surface in this regard. We also made a number of assumptions in the framework which should be noted.

Model Dependence: We used a computational approach and adopted neural networks as our function class. Though we validated the stability of the findings w.r.t various architectures and datasets, it should be noted that the results are in principle model and data specific. The current model also does not include a principled mechanism for handling uncertainty or probabilistic reasoning.

Compositionality: We performed the modeling via a set of common human-defined visual tasks. It is natural to consider a further compositional approach in which such common tasks are viewed as *observed samples* which are composed of computationally found latent (sub)tasks.

Space Regularity: We performed modeling of a dense space via a sampled dictionary. Though we showed a good tolerance w.r.t. to the choice of dictionary and transferring to out-of-dictionary tasks, this outcome holds upon a proper sampling of the space as a function of its regularity. More formal studies on properties of the computed space is required for this to be provably guaranteed for a general case.

Transferring to Non-visual and Robotic Tasks: Given the structure of the space of visual tasks and demonstrated transferabilities to novel tasks, it is worthwhile to question how this can be employed to develop a perception module for solving downstream tasks which are not entirely visual, e.g. robotic manipulation, but entail solving a set of (a priori unknown) visual tasks.

Lifelong Learning: We performed the modeling in one go. In many cases, e.g. lifelong learning, the system is evolving and the number of mastered tasks constantly increase. Such scenarios require augmentation of the structure with expansion mechanisms based on new beliefs.

Acknowledgement: We acknowledge the support of NSF (DMS-1521608), MURI (1186514-1-TBCJE), ONR MURI (N00014-14-1-0671), Toyota(1191689-1-UDAWF), ONR MURI (N00014-13-1-0341), Nvidia, Tencent, a gift by Amazon Web Services, a Google Focused Research Award.

References

[1] P. Agrawal, J. Carreira, and J. Malik. Learning to see by moving. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 37–45, 2015. 8

[2] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, and N. de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems*, pages 3981–3989, 2016. 2

[3] I. Armeni, S. Sax, A. R. Zamir, and S. Savarese. Joint 2d-3d-semantic data for indoor scene understanding. *arXiv preprint arXiv:1702.01105*, 2017. 4

[4] S. Arora, A. Bhaskara, R. Ge, and T. Ma. Provable bounds for learning some deep representations. In *International Conference on Machine Learning*, pages 584–592, 2014. 2

[5] Y. Aytar and A. Zisserman. Tabula rasa: Model transfer for object category detection. In *Computer Vision (ICCV), 2011 IEEE International Conference on*, pages 2252–2259. IEEE, 2011. 2

[6] J. Baxter. A bayesian/information theoretic model of learning to learn viamultiple task sampling. *Mach. Learn.*, 28(1):7–39, July 1997. 3

[7] S. Ben-David and R. S. Borbely. A notion of task relatedness yielding provable multiple-task learning guarantees. *Machine Learning*, 73(3):273–287, Dec 2008. 3

[8] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013. 2

[9] P. Berkhin et al. A survey of clustering data mining techniques. *Grouping multidimensional data*, 25:71, 2006. 2

[10] E. Bienenstock, S. Geman, and D. Potter. Compositionality, mdl priors, and object recognition. In *Advances in neural information processing systems*, pages 838–844, 1997. 2

[11] H. Bilen and A. Vedaldi. Integrated perception with recurrent multi-task neural networks. In *Advances in neural information processing systems*, pages 235–243, 2016. 2

[12] J. Bingel and A. Søgaard. Identifying beneficial task relations for multi-task learning in deep neural networks. *arXiv preprint arXiv:1702.08303*, 2017. 2

[13] O. Boiman and M. Irani. Similarity by composition. In *Advances in neural information processing systems*, pages 177–184, 2007. 2

[14] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Nießner, M. Savva, S. Song, A. Zeng, and Y. Zhang. Matterport3d: Learning from rgb-d data in indoor environments. *arXiv preprint arXiv:1709.06158*, 2017. 4

[15] Z. Chen and B. Liu. *Lifelong Machine Learning*. Morgan & Claypool Publishers, 2016. 2

[16] I. I. CPLEX. V12. 1: User's manual for cplex. *International Business Machines Corporation*, 46(53):157, 2009. 5

[17] C. Doersch, A. Gupta, and A. A. Efros. Unsupervised visual representation learning by context prediction. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1422–1430, 2015. 2

[18] C. Doersch and A. Zisserman. Multi-task self-supervised visual learning. *arXiv preprint arXiv:1708.07860*, 2017. 2

[19] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. In *International conference on machine learning*, pages 647–655, 2014. 2

[20] J. Donahue, P. Krähenbühl, and T. Darrell. Adversarial feature learning. *arXiv preprint arXiv:1605.09782*, 2016. 2

[21] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel. Rl2: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016. 2

[22] D. Erhan, Y. Bengio, A. Courville, P.-A. Manzagol, P. Vincent, and S. Bengio. Why does unsupervised pre-training help deep learning? *Journal of Machine Learning Research*, 11(Feb):625–660, 2010. 2

[23] A. Faktor and M. Irani. "clustering by composition"–unsupervised discovery of image categories. In *European Conference on Computer Vision*, pages 474–487. Springer, 2012. 2

[24] L. Fe-Fei et al. A bayesian approach to unsupervised one-shot learning of object categories. In *Computer Vision, 2003. Proceedings. Ninth IEEE International Conference on*, pages 1134–1141. IEEE, 2003. 2

[25] L. Fei-Fei, R. Fergus, and P. Perona. One-shot learning of object categories. *IEEE transactions on pattern analysis and machine intelligence*, 28(4):594–611, 2006. 2

[26] B. Fernando, A. Habrard, M. Sebban, and T. Tuytelaars. Unsupervised visual domain adaptation using subspace alignment. In *Proceedings of the IEEE international conference on computer vision*, pages 2960–2967, 2013. 2

[27] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. *arXiv preprint arXiv:1703.03400*, 2017. 2

[28] C. Finn, S. Levine, and P. Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. *CoRR*, abs/1603.00448, 2016. 2

[29] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel. Deep spatial autoencoders for visuomotor learning. In *Robotics and Automation (ICRA), 2016 IEEE International Conference on*, pages 512–519. IEEE, 2016. 2

[30] C. Finn, T. Yu, J. Fu, P. Abbeel, and S. Levine. Generalizing skills with semi-supervised reinforcement learning. *CoRR*, abs/1612.00429, 2016. 2

[31] C. Finn, T. Yu, T. Zhang, P. Abbeel, and S. Levine. One-shot visual imitation learning via meta-learning. *CoRR*, abs/1709.04905, 2017. 2

[32] I. K. Fodor. A survey of dimension reduction techniques. Technical report, Lawrence Livermore National Lab., CA (US), 2002. 2

[33] R. M. French. Catastrophic forgetting in connectionist networks: Causes, consequences and solutions. *Trends in Cognitive Sciences*, 3(4):128–135, 1999. 2

[34] R. Ge. *Provable algorithms for machine learning problems*. PhD thesis, Princeton University, 2013. 2

[35] S. Geman, D. F. Potter, and Z. Chi. Composition systems. *Quarterly of Applied Mathematics*, 60(4):707–736, 2002. 2

[36] R. Gopalan, R. Li, and R. Chellappa. Domain adaptation for object recognition: An unsupervised approach. In *Computer Vision (ICCV), 2011 IEEE International Conference on*, pages 999–1006. IEEE, 2011. 2

[37] A. Gopnik, C. Glymour, D. Sobel, L. Schulz, T. Kushnir, and D. Danks. A theory of causal learning in children: Causal maps and bayes nets. 111:3–32, 02 2004. 2

[38] A. Gopnik, C. Glymour, D. M. Sobel, L. E. Schulz, T. Kushnir, and D. Danks. A theory of causal learning in children: causal maps and bayes nets. *Psychological review*, 111(1):3, 2004. 2

[39] A. Gopnik, A. N. Meltzoff, and P. K. Kuhl. *The scientist in the crib: Minds, brains, and how children learn.* William Morrow & Co, 1999. 2

[40] A. Graves, G. Wayne, and I. Danihelka. Neural turing machines. *CoRR*, abs/1410.5401, 2014. 2

[41] I. Gurobi Optimization. Gurobi optimizer reference manual, 2016. 5, 6

[42] K. Henry. The theory and applications of homomorphic cryptography. Master's thesis, University of Waterloo, 2008. 2

[43] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 4

[44] J. Hoffman, T. Darrell, and K. Saenko. Continuous manifold based adaptation for evolving visual domains. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 867–874, 2014. 2

[45] Y. Hoshen and S. Peleg. Visual learning of arithmetic operations. *CoRR*, abs/1506.02264, 2015. 2

[46] F. Hu, G.-S. Xia, J. Hu, and L. Zhang. Transferring deep convolutional neural networks for the scene classification of high-resolution remote sensing imagery. *Remote Sensing*, 7(11):14680–14707, 2015. 2

[47] I.-H. Jhuo, D. Liu, D. Lee, and S.-F. Chang. Robust visual domain adaptation with low-rank reconstruction. In *Computer Vision and Pattern Recognition (CVPR), 2012 IEEE Conference on*, pages 2168–2175. IEEE, 2012. 2

[48] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014. 2

[49] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 2

[50] I. Kokkinos. Ubernet: Training auniversal'convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. *arXiv preprint arXiv:1609.02132*, 2016. 2

[51] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, pages 1097–1105, 2012. 8

[52] B. Kulis, K. Saenko, and T. Darrell. What you saw is not what you get: Domain adaptation using asymmetric kernel transforms. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1785–1792. IEEE, 2011. 2

[53] I. Laina, C. Rupprecht, V. Belagiannis, F. Tombari, and N. Navab. Deeper depth prediction with fully convolutional residual networks. In *3D Vision (3DV), 2016 Fourth International Conference on*, pages 239–248. IEEE, 2016. 7

[54] B. M. Lake, R. Salakhutdinov, and J. B. Tenenbaum. Human-level concept learning through probabilistic program induction. *Science*, 350(6266):1332–1338, 2015. 2

[55] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman. Building machines that learn and think like people. *Behavioral and Brain Sciences*, pages 1–101, 2016. 2

[56] Y. Li, H. Qi, J. Dai, X. Ji, and Y. Wei. Fully convolutional instance-aware semantic segmentation. *arXiv preprint arXiv:1611.07709*, 2016. 4

[57] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014. 4

[58] F. Liu, G. Lin, and C. Shen. CRF learning with CNN features for image segmentation. *CoRR*, abs/1503.08263, 2015. 2

[59] Z. Luo, Y. Zou, J. Hoffman, and L. F. Fei-Fei. Label efficient learning of transferable representations acrosss domains and tasks. In *Advances in Neural Information Processing Systems*, pages 164–176, 2017. 2

[60] M. M. H. Mahmud. *On Universal Transfer Learning*, pages 135–149. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. 3

[61] J. Malik, P. Arbeláez, J. Carreira, K. Fragkiadaki, R. Girshick, G. Gkioxari, S. Gupta, B. Hariharan, A. Kar, and S. Tulsiani. The three r's of computer vision: Recognition, reconstruction and reorganization. *Pattern Recognition Letters*, 72:4–14, 2016. 2

[62] N. Masuda, M. A. Porter, and R. Lambiotte. Random walks and diffusion on networks. *Physics Reports*, 716-717:1 – 58, 2017. Random walks and diffusion on networks. 5

[63] M. Mccloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24, 1989. 2

[64] L. Mihalkova, T. Huynh, and R. J. Mooney. Mapping and revising markov logic networks for transfer learning. In *AAAI*, volume 7, pages 608–614, 2007. 2

[65] T. Mikolov, Q. V. Le, and I. Sutskever. Exploiting similarities among languages for machine translation. *CoRR*, abs/1309.4168, 2013. 2

[66] I. Misra, A. Shrivastava, A. Gupta, and M. Hebert. Cross-stitch networks for multi-task learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3994–4003, 2016. 2

[67] A. Niculescu-Mizil and R. Caruana. Inductive transfer for bayesian network structure learning. In *Artificial Intelligence and Statistics*, pages 339–346, 2007. 2

[68] M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *European Conference on Computer Vision*, pages 69–84. Springer, 2016. 2, 8

[69] M. Noroozi, H. Pirsiavash, and P. Favaro. Representation learning by learning to count. *arXiv preprint arXiv:1708.06734*, 2017. 2

[70] M. Norouzi, T. Mikolov, S. Bengio, Y. Singer, J. Shlens, A. Frome, G. S. Corrado, and J. Dean. Zero-shot learning by convex combination of semantic embeddings. *arXiv preprint arXiv:1312.5650*, 2013. 2

[71] M. Ovsjanikov, M. Ben-Chen, J. Solomon, A. Butscher, and L. Guibas. Functional maps: a flexible representation of maps between shapes. *ACM Transactions on Graphics (TOG)*, 31(4):30, 2012. 2

[72] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2536–2544, 2016. 2

[73] A. Pentina and C. H. Lampert. Multi-task learning with labeled and unlabeled tasks. *stat*, 1050:1, 2017. 2

[74] J. Piaget and M. Cook. *The origins of intelligence in children*, volume 8. International Universities Press New York, 1952. 2

[75] L. Y. Pratt. Discriminability-based transfer between neural networks. In *Advances in neural information processing systems*, pages 204–211, 1993. 2

[76] S. R. Richter, Z. Hayder, and V. Koltun. Playing for benchmarks. In *International Conference on Computer Vision (ICCV)*, 2017. 2

[77] S. T. Roweis and L. K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *science*, 290(5500):2323–2326, 2000. 2

[78] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015. 2, 4, 8, 9

[79] R. W. Saaty. The analytic hierarchy process – what it is and how it is used. *Mathematical Modeling*, 9(3-5):161–176, 1987. 5

[80] K. Saenko, B. Kulis, M. Fritz, and T. Darrell. Adapting visual category models to new domains. *Computer Vision–ECCV 2010*, pages 213–226, 2010. 2

[81] R. Salakhutdinov, J. Tenenbaum, and A. Torralba. One-shot learning with a hierarchical nonparametric bayesian model. In *Proceedings of ICML Workshop on Unsupervised and Transfer Learning*, pages 195–206, 2012. 2

[82] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. Trust region policy optimization. *CoRR*, abs/1502.05477, 2015. 2

[83] A. Sharif Razavian, H. Azizpour, J. Sullivan, and S. Carlsson. Cnn features off-the-shelf: an astounding baseline for recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 806–813, 2014. 2

[84] D. L. Silver and K. P. Bennett. Guest editor's introduction: special issue on inductive transfer learning. *Machine Learning*, 73(3):215–220, 2008. 2

[85] D. L. Silver, Q. Yang, and L. Li. Lifelong machine learning systems: Beyond learning algorithms. In *in AAAI Spring Symposium Series*, 2013. 2

[86] R. Socher, M. Ganjoo, C. D. Manning, and A. Ng. Zero-shot learning through cross-modal transfer. In *NIPS*, pages 935–943, 2013. 2

[87] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014. 2

[88] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus. Intriguing properties of neural networks. *CoRR*, abs/1312.6199, 2013. 2

[89] J. B. Tenenbaum and T. L. Griffiths. Generalization, similarity, and bayesian inference. *Behavioral and Brain Sciences*, 24(4):629–640, 2001. 2

[90] J. B. Tenenbaum, C. Kemp, T. L. Griffiths, and N. D. Goodman. How to grow a mind: Statistics, structure, and abstraction. *science*, 331(6022):1279–1285, 2011. 2

[91] J. B. Tenenbaum, C. Kemp, and P. Shafto. Theory-based bayesian models of inductive learning and reasoning. In *Trends in Cognitive Sciences*, pages 309–318, 2006. 2

[92] D. G. R. Tervo, J. B. Tenenbaum, and S. J. Gershman. Toward the neural implementation of structure learning. *Current opinion in neurobiology*, 37:99–105, 2016. 2

[93] C. Tessler, S. Givony, T. Zahavy, D. J. Mankowitz, and S. Mannor. A deep hierarchical approach to lifelong learning in minecraft. In *AAAI*, pages 1553–1561, 2017. 2

[94] S. Thrun and L. Pratt. *Learning to learn*. Springer Science & Business Media, 2012. 2

[95] A. M. Turing. Computing machinery and intelligence. *Mind*, 59(236):433–460, 1950. 2

[96] X. Wang and A. Gupta. Unsupervised learning of visual representations using videos. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2794–2802, 2015. 8

[97] X. Wang, K. He, and A. Gupta. Transitive invariance for self-supervised visual representation learning. *arXiv preprint arXiv:1708.02901*, 2017. 2

[98] T. Winograd. *Thinking machines: Can there be? Are we*, volume 200. University of California Press, Berkeley, 1991. 2

[99] J. Yang, R. Yan, and A. G. Hauptmann. Adapting svm classifiers to data with shifted distributions. In *Data Mining Workshops, 2007. ICDM Workshops 2007. Seventh IEEE International Conference on*, pages 69–76. IEEE, 2007. 2

[100] A. R. Zamir, T. Wekel, P. Agrawal, C. Wei, J. Malik, and S. Savarese. Generic 3d representation via pose estimation and matching. In *European Conference on Computer Vision*, pages 535–553. Springer, 2016. 2, 8

[101] A. R. Zamir, F. Xia, J. He, A. Sax, J. Malik, and S. Savarese. Gibson Env: Real-world perception for embodied agents. In *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2018. 4

[102] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. *CoRR*, abs/1611.03530, 2016. 2

[103] R. Zhang, P. Isola, and A. A. Efros. Colorful image colorization. In *European Conference on Computer Vision*, pages 649–666. Springer, 2016. 2, 8

[104] B. Zhou, A. Lapedriza, J. Xiao, A. Torralba, and A. Oliva. Learning deep features for scene recognition using places database. In *Advances in neural information processing systems*, pages 487–495, 2014. 2, 4, 8, 9

Taskonomy: Disentangling Task Transfer Learning
Supplementary Material

Amir R. Zamir[1,2] Sasha Sax[1*] William Shen[1*] Leonidas Guibas[1] Jitendra Malik[2] Silvio Savarese[1]

[1] Stanford University [2] University of California, Berkeley

http://taskonomy.stanford.edu/

Abstract

The following items are provided in the supplementary material:

1. *Higher-order transfer experiments.*
2. *A tiny-data taxonomy.*
3. *Optimal transfer policies for selected tasks.*
4. *Our networks as feature extractors on MIT Places.*
5. *BIP Solver as API.*
6. *Significance testing of taxonomy.*
7. *Qualitative results of network output.*
8. *Agglomerative clustering of tasks.*
9. *Analysis of transitive transfers.*
10. *Experimental setup details.*
11. *Task dictionary definition and discussion.*

1. Higher-Order Transfers

In the main paper, low-order transfers (orders 2-4) often achieved superior performance to first-order transfers. What happens as we continue to increase the order? Do we get the best performance from using the entire bank of task-specific networks?

In addition to training many first-order and low-order (orders 2-4) transfers, we trained one transfer function for each order 5-25. Each high-order transfer function was trained on the combined representations of the top k first-order transfers. The high-order transfers are available in the provided API. The calculated distances matrix including higher-order transfers is shown in figure 2.

2. Tiny Data Taxonomy

As the amount of supervision of the transfer networks increases, the performance margin between them decreases. Eventually, full supervision is possible. Clearly, the amount of data available for training transfers is an important factor.



Figure 1: **Selected policies.** For tasks in the table form Figure 10 of the main paper, we show the best policies for transferring to that task. The center node of each cluster is the target task, and the surrounding nodes at the tail of each arrow are the suggested source tasks.

We present a low-data taxonomy experiment in which the transfer networks were trained on 1k data points, as opposed to the main experiments which used 16k data points. The distance matrix for these transfers is in figure 3, and the transfers themselves are accessible in the provided API.

*equal.

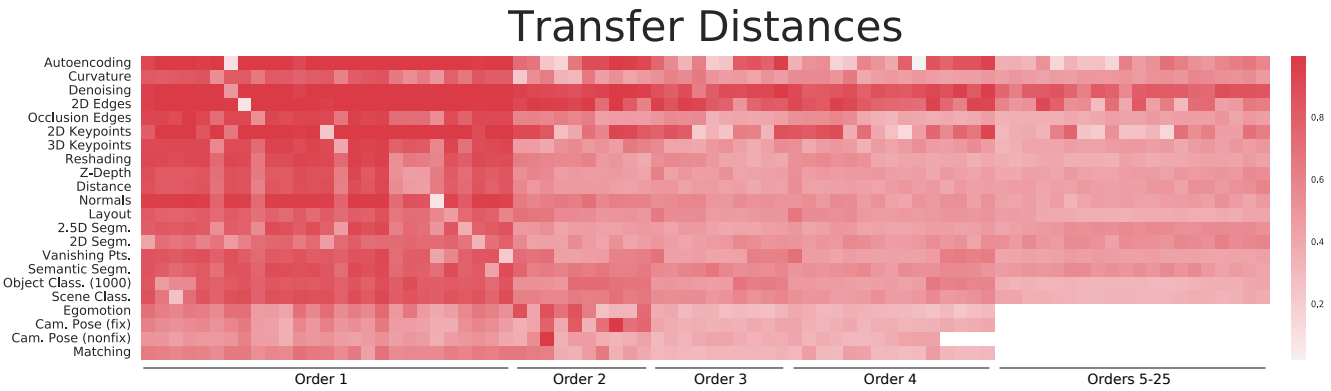


Figure 2: **Higher Order Distances.** Calculated task distances for all 16k transfers. The columns are sorted by transfer order and the orders are shown along the x-axis. Each row is a target task.

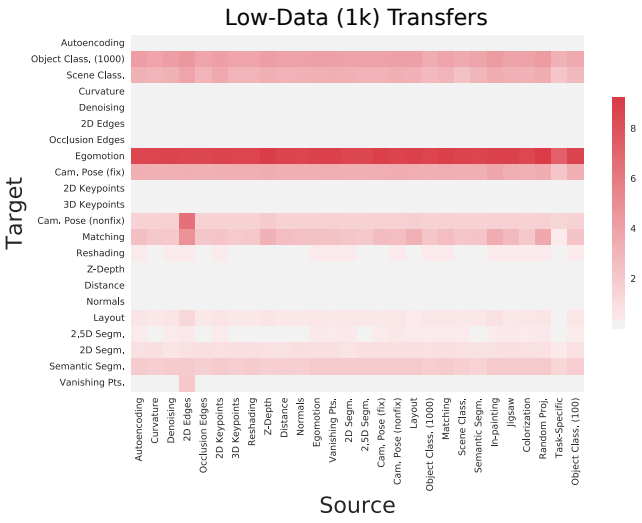


Figure 3: **Low data transfers.** Expected loss after training transfer functions with 1k samples (shown before AHP normalization). This is a low-data version of the transfer loss matrix in the main paper.

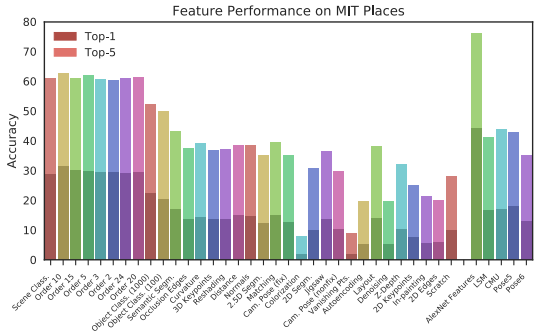


Figure 4: **Feature extraction.** Performance of readout functions trained on MIT Places. We compare our networks to common baselines. The x-axis is ordered according to transfer performance on our dataset, and the y-axis shows Top-1 and Top-5 accuracy on MIT Places. The sources on the left were trained on our dataset, while the right portion contains external baselines.

3. Special Task Policies

In the main paper, we showed the best transfer performance of our networks against some commonly used baselines. In figure 1, we now show the collections of transfers that beat the baselines shown in figure 9 in the main paper. In some cases, higher-order transfers outperformed the ones in figure 9, and we show those instead.

4. Feature Performance on MIT Places

When data labels are scarce, AlexNet and other pre-trained networks are commonly used as feature extractors that provide input for a small readout function. We compare the performance of our task-specific features to AlexNet features on 16k images from the 63 selected classes on the MIT Places dataset. While AlexNet was trained on 1M hand-labeled images, our task-specific networks were trained on 120k images where labels were generated automatically. The final results of the MIT feature extraction experiment are shown in figure 4.

5. BIP Solver as API

The BIP solver allows users to query our results and quickly compute the best transfer taxonomy for their particular use case. In this sense it can acts as an API, and we refer to it as the Transfer Learning API, or just the API. http://taskonomy.stanford.edu/api provides access to this API via a webpage and allows interactively entering the arguments and computing the corresponding transfer taxonomy. Qualitative transfer results corresponding to the computed taxonomy exemplified over a YouTube video is also shown.

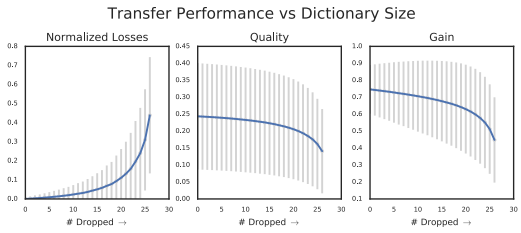


Figure 5: **Drop-k performance.** Expected performance of transferring to a task after dropping k tasks from our dictionary. The grey bars show the standard deviation of the performance.

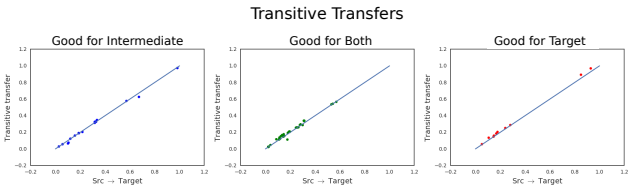


Figure 6: **Transitivity.** Loss after transitive transfer is on the y-axis and loss after direct ($A \rightarrow C$) transfer is on the x-axis. The line x=y is shown. Red dots are transfers which transferred well to C but not to B in first-order. Blue dots transferred well to the target, C, but not to B. Green dots transferred well to both.

6. Sensitivity to Choice of Dictionary

How universal are our findings with respect to the modeling dictionary? Is our coverage of the task space sufficiently dense? We evaluate this by randomly dropping some tasks from the dictionary and evaluating first-order transfer performance to tasks, and averaging over tasks. If our dictionary is dense, we should see a gradual drop in performance until our dictionary becomes too sparse, at which point performance will degrade rapidly. Results are shown in figure 5.

Additionally, in order to give a more comprehensive idea of the opportunity costs of choosing sub-optimal transfers, we show the losses, quality, and gain of all first-order transfers in figure 8.

7. Transfer via Transitivity

Perhaps transfer can be done in a transitive manner: first learn task A from images, and then B from A, and finally C from B. Intuitively, the performance $A \rightarrow C$ might improve if existing neural network machinery can take advantage of the additional computational power and supervision in the transitive $A \rightarrow B \rightarrow C$.

See figure 6 for the a comparison of the two transfer types. Based on the intermediate transfer B, we show three cases for transitive transfers: B transferred well to only A, only C, or both A and C. We found that the loss for transfers $A \rightarrow B \rightarrow C$ is very close to the loss for $A \rightarrow C$. This means that the largest contributor to performance on C is the representations learned for A, and that neither a doubling of the transfer computation nor the additional supervision for a proxy task has a strong effect on the performance. Since these results suggest that NNs are unable to effectively use transitive transfers, even if we modeled them, we are enforcing the taxonomy to be free of chains longer than length 1.

8. I. Model Specificness.

In principle, the findings are model dependent as a natural consequence of employing a fully computational formulation. However, we performed a set of experiments with varying transfer network and task-specific network architectures to quantify model independence within the function class of neural nets. The AHP affinity matrices turned out to be very similar (left) and the variance matrices (right) show pretty low values, signifying a high invariance.

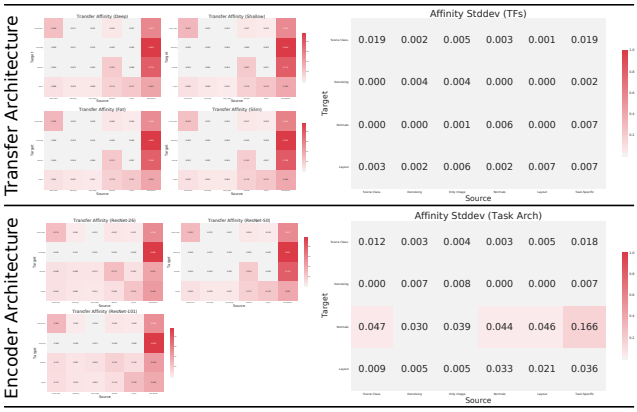


Figure 7: **Model Specificity.** (left) Affinity matrices. (right) Standard Deviation of Affinities.

9. Comparison to per-task state-of-the-art methods

We reported the win rate of our task specific models vs. two statistical baselines of average estimator *avg* (the best statistically informed guess) and a supervised readout network trained on a random (Gaussian) network's representation (*rand*) in the main paper. To get a sense of the quality of our networks vs. state-of-the-art task-specific methods, we compared our depth estimator vs. released models of Fully Convolutional Residual Network [20]. The results are provided in the table below showing our task specific network outperformed [20]. In general, we found the task-specific networks to perform on par or better than state-of-the-art for many of the tasks, though we do not formally benchmark or claim this.

Evaluation Pair	Win Rate	Loss
Ours (120k training images) v FCRN	80%	**0.39** v 0.47
Ours (3M training images) v FCRN	88%	**0.35** v 0.47

10. Evaluation on MIT Places & ImageNet: Experimental Details

Does our ranking of transfers hold across datasets? Is the taxonomy ordering something innate to the tasks, or an artifact of our data? We evaluated this on MIT Places and ImageNet, and found a very strong correlation between our ordering and the one on the new dataset; the results are reported in the main paper. The setup for these experiments is as follows:

Transfer to MIT Places: We evaluated the order on Places-365 in the following way. For each task in our dataset, we reinitialized a new transfer function, attached it to the task-specific encoder, and trained the network normally on 16k images from the 63 classes in the Home+Hotel and Work subgroups from Places-365, classes which were likely to be present in our dataset.

Note that the places training regimen is different from the one used to form the taxonomy, and the training and evaluation occur on different datasets. Despite this, correlation between the two orders was very high.

Transfer to ImageNet: Similar to our Places experiment, we selected a subset of categories likely to appear in our dataset (e.g. appliances, furniture). This had little effect on the ordering on our dataset. Then we distilled a trained ResNet-152 [12] and train a transfer function to predict the activations.

11. Baselines

To evaluate the performance of our networks, we compare our performance against 6 common baselines. Four of the baselines are trained on different external datasets:

1. **ImageNet fc7[19]**, with representation size 4096, is trained on ImageNet [34] 1000l way classification.
2. **Wang et al. [43]**, with representation size 1024, is trained on 100K videos from YouTube for unsupervised patch triplet ranking.
3. **Agrawal et al. [2]**, with representation shape $13 \times 13 \times 256$, is an unsupervised method trained on KITTI to do egomotion in outdoor environments.
4. **Zamir et al. [49]**, with representation size 500, is trained to do both relative camera pose prediction and point matching.

Two of the baselines are trained as standard tasks on our dataset: Colorization [50] and Context Prediction (Jigsaw) [30], and details are defined in sec. 14.

In the comparison to baseline tasks shown in Figure 10 of the main paper, the experimental setup was as follows: we extracted features for each baseline network, and then trained transfer functions to predict the labels of other tasks using the extracted representations. In the case where the baseline features were not of size 2048, we added an extra fully-connected layer. We then evaluated the quality and gain of our networks over these baselines using our standard ordinal metric.

We also compare our networks to these baselines on other datasets for which our networks were not trained. We compare the features extracted by our networks to those extracted by the baselines on MIT Places [52] by training readout functions. Results are shown in 4.

12. Network Architecture

We used Tensorflow for our implementation.

Encoder: For all tasks we modified a ResNet-50 encoder with no average-pooling and replace the last stride 2 convolution with stride 1. This gives us an output shape of $16 \times 16 \times 2048$; we use a 3×3 convolution to transform the output to our final representation, which is of shape $16 \times 16 \times 8$.

Decoder: The shape of the decoder depends on the task. For pixels-to-pixels prediction, we used a 15-layer fully convolutional decoder: 5 convolutional layers, and then alternating convolution and convolution_transpose layers. For low-dimensional tasks such as Vanishing Points, our decoder was two or three fully-connected layers with dropout applied after all but the last one. For all layers (except output), we used batchnorm.

Transfer Function: Our transfer functions have a tower structure, and multiple towers have their output concatenated channelwise. Our towers are feedforward with the following set of functions: first clip extreme values to 5-sigma and then do batch renormalization, followed by two convolutional layers. In the case where the readout function takes an image as input, these layers are stride-4 dilated convolutions. As our transfer functions may use both an image and a representation, we used two towers in every transfer function – if there was no input image, we fed a representation to both towers.

Loss Functions: For pixels-to-pixels prediction tasks we used a CGAN in addition to the standard loss function. The GAN training began after 25000 steps, and the discriminator was 5 layers with $10\times$ the standard weight decay. The loss was a linear combination of the $0.996\times$ the standard loss and $0.004\times$ the GAN loss. We found that including even this small amount of GAN loss helped to sharpen the images.

Hyperparameters: We used ADAM with a learning rate of 10^{-4} and annealed the learning rate by a factor of 10 after 80000 steps. We used batch size 32 and weight decay

2×10^{-6}.

Data: Out of more than 1M images collected, we used 120k from 87 buildings to train task-specific networks. The transfer networks were trained on 16k images from a separate 24 buildings. The taxonomy was generated from these transfer networks on a third set of 17k images from 13 buildings.

13. Pseudo-semantics Annotations

As we do not have semantic annotations on our dataset, we gathered pseudo-semantic annotations by using knowledge distillation [14] approach. That is, we labeled our dataset using the output of state-of-the-art large fully-supervised network (ResNet-151) for semantic objects (using ImageNet, 100 applicable classes), scene categories (using MIT Places [52], 63 indoor workplace and home classes in MIT Places scene hierarchy), and semantic object segmentation (using COCO [22], 17 applicable classes). In a user study, human annotators indicated that only 6.3% of images have no plausible label in their top-5 classes in Scene categories, showing that such labels are reliable enough and effective for being a gateway to modeling semantics in task space. The quality of the semantic labels can be seen in the provided sample of qualitative results or in the video. The list of selected classes for COCO and ImageNet are available here.

14. Detailed Definition of Task Dictionary

Task definitions are presented in alphabetical order:

Autoencoding PCA is a widely used method of understanding data by finding a low-dimensional latent representation. Autoencoding [15] is a nonlinear generalization of PCA that was originally proposed with transfer learning in mind: better downstream performance through autoencoding pretraining.

Colorization [50] Colorization requires taking a grayscale image and predicting the original colors. It is an unsupervised task, but also one that is semantic-aware[6]. For example, predicting the color of a fruit is simple once the fruit is identified.

Context Encoding Context Encoding was first introduced by Pathak et al. [31] and is a version of autoencoding where a large portion of the input is masked from the model. In order to fill in the occluded area, the model must reason about the scene geometry and semantics. Similar to colorization, it is an unsupervised-yet-still-semantic task.

Content Prediction (Jigsaw)[30] A discriminative version of Context Encoding, Jigsaw [30] requires a network to unscramble a permuted tiling of the input image.

Curvature Estimation Curvature-based features are excellent for identification because they are invariant under rigid transformations. Curvature is known to be important in visual processing–so much so that the Macaque visual cortex has a dedicated curvature processing region [47].

Denoising It is desirable for similar inputs to have similar representations, but representations learned by autoencoding are excessively sensitive to perturbations in the input. Denoising [41] (autoencoding) encourages limited invariance by mapping slightly perturbed inputs to the unperturbed input.

Depth Estimation, Euclidean Depth estimation is an important task, useful for detecting proximity to obstacles and items of interest. It is also a useful intermediate step for agents to localize themselves in 3D space. Euclidean depth is measured as the distance from each pixel to the camera's optical center.

Depth Estimation, Z-Buffer As opposed to Euclidean depth estimation, researchers typically use z-buffer depth which is defined as the distance to the camera plane. This is not the way that humans typically perceive depth, but is included because this is the standard formulation and all of our depth-derived tasks are derived from *z-buffer*.

Edge Detection (2D) Edge detection is historically a fundamental task in computer vision. Edges are commonly used as an intermediate representation or as a feature in a larger processing pipeline. We include the output of a Canny edge detector without nonmax suppression (to make the task learnable by neural networks).

Edge Detection (3D) As opposed to 2D edges, we define 3D edges as "occlusion edges," or edges where an object in the foreground obscures something behind it. 2D edges respond to changes in texture, but 3D edges are features which depend only on the 3D geometry and are invariant to color and lighting.

Keypoint Detection (2D) Keypoint detection has a long history in computer vision, and is useful for many, many tasks. Keypoint algorithmns usually consist of two parts, both a keypoint detector and some local patch descriptor which is invariant across multiple images [25, 4, 33]. 2D keypoint detection encourages the network to identify locally important regions of an image, and *point matching* encourages the network to learn feature descriptors. Identifying keypoints is frequently still a first step in a larger visual pipeline. We use the output of SURF [4] (before nonmax suppression) as our ground-truth.

Keypoint Detection (3D) 3D keypoints are similar to 2D keypoints except that they are derived from 3D data and therefore account for scene geometry. They are often invariant to informative (but possibly distracting) features such as textures [51, 40, 26, 48, 17]. We use the output of the NARF [40] algorithm (before nonmax suppression) as our 3D keypoint ground-truth.

Point Matching Deep networks trained for point matching learn feature descriptors that prove useful for downstream tasks. Point matching has applications in fine-

grained classification [45] and object recognition [24], multi-view reconstruction [37] and structure from motion [28], wide baseline matching [42], SLAM [36] and visual odometry[53].

Relative Camera Pose Estimation, Non-Fixated The famous "Kitten Carousel" experiment by Held and Hein [13] suggested that taking action is crucial for strong perception. Although more recent works call the original conclusion into question [32], the ability to localize oneself remains important for locomotion. For two different views with the same optical centers, we try to predict the 6-DOF relative camera pose (yaw, pitch, roll, xyz translation) between them.

Relative Camera Pose Estimation, Fixated We also include a simpler variant of camera pose estimation for which the center pixel of the two inputs is always the same physical 3D point. This problem is simpler in the sense that there are only five degrees of freedom.

Relative Camera Pose Estimation, Triplets (Egomotion) Videos are a common object of study in computer vision (e.g. visual odometry [10, 29]) and they provide dense data with high redundancy. We therefore include camera pose matching for input triplets with a fixed center point. With three images, models have a greater ability to match points for accurate localization.

Reshading One way to infer scene geometry is "shape from shading" [3] using the intrinsic image decomposition $I = A \cdot S$, where S is a shading function parameterized by lighting and depth. This decomposition is thought to be useful in human visual perception [1]. We define reshading as follows: Given an RGB image, the label is the shading function S that results from having a single point light at the camera origin, and S is multiplied by a constant fixed albedo.

Room Layout Estimation Estimating and aligning a 3D bounding box is a mid-level task that includes vanishing point estimation as a sub-problem, and has applications for robotic navigation [39], scene reconstruction [16], and augmented reality [9]. A variant of room layout estimation was used in the LSUN room layout challenge [46], but that formulation is ill-posed when there is camera roll or when no room corners are in view. Instead, we offer a formulation that remains well-defined regardless of the camera pose and field of view. This task includes some semantic information such as 'what constitutes a room' while simultaneously also including scene geometry.

Segmentation, Unsupervised (2D)) Gestalt psychologists proposed principles of grouping as a mechanism through which humans learn to perceive the world as a set of coherent objects [44]. Normalized cuts [38] are one method for segmenting images into perceptually similar groups, and we include this Gestalt task in our dictionary.

Segmentation, Unsupervised (2.5D)) Segmentation 2.5D uses the same algorithm as 2D, but the labels are computed jointly from the RGB image, the aligned depth image, and the aligned surface normals image. Therefore the 2.5D segmentation applies the principles not just to the world as it seems (in the RGB image), but also to the world as it is (ground-truth 3D). 2.5D segmentation incorporates information about the scene geometry that is not directly present in the RGB image but that is readily inferred by humans.

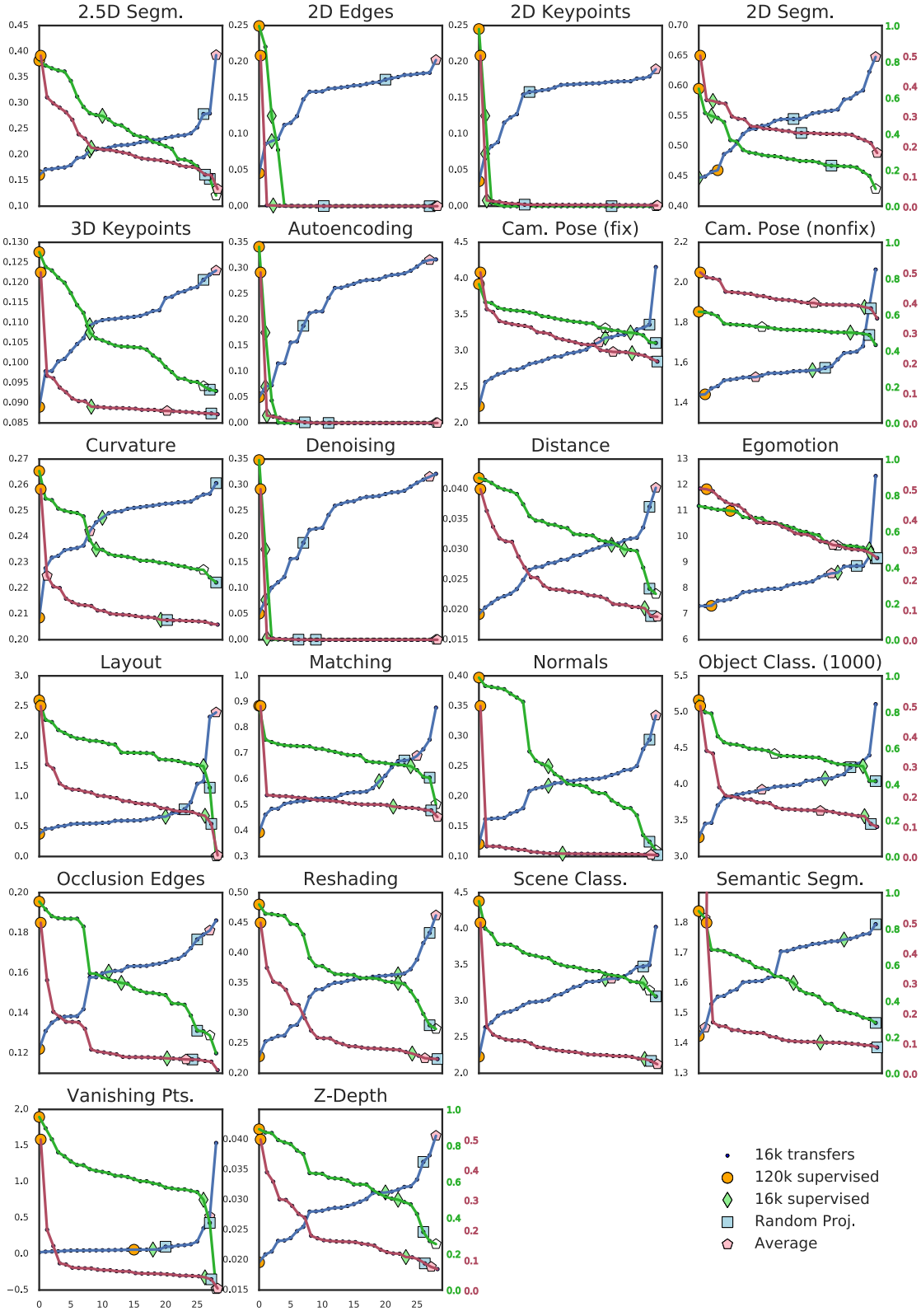
Surface Normal Estimation Surface normal estimation is thought to be crucial for spatial cognition. For example, objects can only be placed on surfaces with upwards-facing normals. Even for locomotion, a point with horizontal-facing normals indicates that it cannot be easily traversed. Surface normals are computed directly from the 3D mesh.

Vanishing Point Estimation An consequence of perspective, vanishing points offer useful information about the scene geometry [27, 18] and are well studied. Vanishing points prove particularly useful in a Manhattan world [8, 50, 5] where there are three dominant vanishing points corresponding to an X, Y, and Z axis. This assumption is usually met in urban environments. For each model we analytically find these three vanishing points and include them as labels.

Semantic Learning through Knowledge distillation While our dataset does not include semantic annotations, semantic understanding comprises a large and important component of modern computer vision. Therefore, we add pseudo-semantic annotations through knowledge distillation [14]. We distill knowledge from state-of-the-art models [11, 21] trained on ImageNet [34] and MS-COCO [23] by using them to annotate our dataset and then supervising models with those annotations. See section 13 for the details of knowledge distillation process.

1. **Classification, Semantic (1000-classes)** Semantic object recognition is a fundamental component of visual perception. Children learn at an early age to classify objects after seeing just a handful of examples [7]. For semantic classification, we distill knowledge form a pretrained ResNet-152 [11] (trained on ImageNet) by supervising our model with the ResNet activations.
2. **Classification Semantic (100-classes)** Many of the classes in ImageNet never appear in our dataset (e.g. animals, sports). We therefore manually select classes which appear in our dataset – and this happened to be 100 classes. For 100-way classification, we distill knowledge from these 100 activations only.
3. **Segmentation, Semantic** Models and agents need to know more than just what they are looking at – they also need to be able to locate the object. Therefore, we include knowledge distillation from the semantic segmentation model in [21], trained on MS COCO. [35]

Sorted **Quality**, **Gain**, and **Loss** of 1st order transfers for each task



References

[1] E. H. Adelson and A. P. Pentland. The perception of shading and reflectance. *Perception as Baysian Inference*, pages pp. 409–423, 1996. 6

[2] P. Agrawal, J. Carreira, and J. Malik. Learning to see by moving. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 37–45, 2015. 4

[3] J. T. Barron and J. Malik. Shape, illumination, and reflectance from shading. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(8):1670–1687, Aug 2015. 6

[4] H. Bay, T. Tuytelaars, and L. Van Gool. *SURF: Speeded Up Robust Features*, pages 404–417. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006. 5

[5] J. C. Bazin, Y. Seo, C. Demonceaux, P. Vasseur, K. Ikeuchi, I. Kweon, and M. Pollefeys. Globally optimal line clustering and vanishing point estimation in manhattan world. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pages 638–645, June 2012. 6

[6] A. Y.-S. Chia, S. Zhuo, R. K. Gupta, Y.-W. Tai, S.-Y. Cho, P. Tan, and S. Lin. Semantic colorization with internet images. *ACM Trans. Graph.*, 30(6):156:1–156:8, Dec. 2011. 5

[7] E. M. Clerkin, E. Hart, J. M. Rehg, C. Yu, and L. B. Smith. Real-world visual statistics and infants' first-learned object names. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 372(1711):20160055, 2017. 6

[8] J. M. Coughlan and A. L. Yuille. The manhattan world assumption: Regularities in scene statistics which enable bayesian inference. In T. K. Leen, T. G. Dietterich, and V. Tresp, editors, *Advances in Neural Information Processing Systems 13*, pages 845–851. MIT Press, 2001. 6

[9] D. DeTone, T. Malisiewicz, and A. Rabinovich. Deep image homography estimation. *CoRR*, abs/1606.03798, 2016. 6

[10] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012. 6

[11] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015. 6

[12] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 4

[13] R. Held and A. Hein. Movement-produced stimulation in the development of visually guided behavior. *J Comp Physiol Psychol*, pages 872–876. 6

[14] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 5, 6

[15] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. 5

[16] H. Izadinia, Q. Shan, and S. M. Seitz. IM2CAD. *CoRR*, abs/1608.05137, 2016. 6

[17] J. Knopp, M. Prasad, G. Willems, R. Timofte, and L. Van Gool. *Hough Transform and 3D SURF for Robust Three Dimensional Classification*, pages 589–602. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010. 5

[18] H. Kong, J. Y. Audibert, and J. Ponce. Vanishing point detection for road detection. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 96–103, June 2009. 6

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, pages 1097–1105, 2012. 4

[20] I. Laina, C. Rupprecht, V. Belagiannis, F. Tombari, and N. Navab. Deeper depth prediction with fully convolutional residual networks. In *3D Vision (3DV), 2016 Fourth International Conference on*, pages 239–248. IEEE, 2016. 3

[21] Y. Li, H. Qi, J. Dai, X. Ji, and Y. Wei. Fully convolutional instance-aware semantic segmentation. *arXiv preprint arXiv:1611.07709*, 2016. 6

[22] T. Lin, M. Maire, S. J. Belongie, L. D. Bourdev, R. B. Girshick, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: common objects in context. *CoRR*, abs/1405.0312, 2014. 5

[23] T. Lin, M. Maire, S. J. Belongie, L. D. Bourdev, R. B. Girshick, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: common objects in context. *CoRR*, abs/1405.0312, 2014. 6

[24] D. G. Lowe. Object recognition from local scale-invariant features. In *Proceedings of the Seventh IEEE International Conference on Computer Vision*, volume 2, pages 1150–1157 vol.2, 1999. 6

[25] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *Int. J. Comput. Vision*, 60(2):91–110, Nov. 2004. 5

[26] A. Mian, M. Bennamoun, and R. Owens. On the repeatability and quality of keypoints for local feature-based 3d object retrieval from cluttered scenes. *International Journal of Computer Vision*, 89(2):348–361, Sep 2010. 5

[27] O. Miksik. Rapid vanishing point estimation for general road detection. In *2012 IEEE International Conference on Robotics and Automation*, pages 4844–4849, May 2012. 6

[28] N. D. Molton, A. J. Davison, and I. D. Reid. Locally planar patch features for real-time structure from motion. In *Proc. British Machine Vision Conference*. BMVC, Sept. 2004. (To appear). 6

[29] D. Nister, O. Naroditsky, and J. Bergen. Visual odometry. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, volume 1, pages I–652–I–659 Vol.1, June 2004. 6

[30] M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *European Conference on Computer Vision*, pages 69–84. Springer, 2016. 4, 5

[31] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. Efros. Context encoders: Feature learning by inpainting. 2016. 5

[32] N. Rader, M. Bausano, and J. E. Richards. On the nature of the visual-cliff-avoidance response in human infants. *Child Development*, 51(1):61–68, 1980. 6

[33] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski. Orb: An efficient alternative to sift or surf. In *Proceedings of the 2011 International Conference on Computer Vision*, ICCV '11, pages 2564–2571, Washington, DC, USA, 2011. IEEE Computer Society. 5

[34] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. 4, 6

[35] A. Sax*, W. B. Shen*, A. R. Zamir*, L. J. Guibas, J. Malik, and S. Savarese. Taskonomy: Disentangling task transfer learning. *CVPR*, 2017. submitted. 6

[36] S. Se, D. Lowe, and J. Little. Mobile robot localization and mapping with uncertainty using scale-invariant visual landmarks, 2002. 6

[37] S. M. Seitz, B. Curless, J. Diebel, D. Scharstein, and R. Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 1, pages 519–528, June 2006. 6

[38] J. Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.*, 22(8):888–905, Aug. 2000. 6

[39] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus. *Indoor Segmentation and Support Inference from RGBD Images*, pages 746–760. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. 6

[40] B. Steder, R. B. Rusu, K. Konolige, and W. Burgard. Narf: 3d range image features for object recognition. 5

[41] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th International Conference on Machine Learning*, ICML '08, pages 1096–1103, New York, NY, USA, 2008. ACM. 5

[42] L. Wang, U. Neumann, and S. You. Wide-baseline image matching using line signatures. In *ICCV*, pages 1311–1318. IEEE Computer Society, 2009. 6

[43] X. Wang and A. Gupta. Unsupervised learning of visual representations using videos. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2794–2802, 2015. 4

[44] M. Wertheimer. Laws of organization in perceptual forms. *Psycologische Forschung*, 4:301–350, 1923. 6

[45] B. Yao, G. Bradski, and L. Fei-Fei. A codebook-free and annotation-free approach for fine-grained image categorization. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3466–3473, June 2012. 6

[46] F. Yu, Y. Zhang, S. Song, A. Seff, and J. Xiao. Lsun: Construction of a large-scale image dataset using deep learning with humans in the loop. *arXiv preprint arXiv:1506.03365*, 2015. 6

[47] X. Yue, I. S. Pourladian, R. B. H. Tootell, and L. G. Ungerleider. Curvature-processing network in macaque visual cortex. *Proceedings of the National Academy of Sciences*, 111(33):E3467–E3475, 2014. 5

[48] A. Zaharescu, E. Boyer, K. Varanasi, and R. Horaud. Surface feature detection and description with applications to mesh matching. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 373–380, June 2009. 5

[49] A. R. Zamir, T. Wekel, P. Agrawal, C. Wei, J. Malik, and S. Savarese. Generic 3d representation via pose estimation and matching. In *European Conference on Computer Vision*, pages 535–553. Springer, 2016. 4

[50] L. Zhang, H. Lu, X. Hu, and R. Koch. Vanishing point estimation and line classification in a manhattan world with a unifying camera model. *International Journal of Computer Vision*, 117(2):111–130, Apr 2016. 4, 5, 6

[51] Y. Zhong. Intrinsic shape signatures: A shape descriptor for 3d object recognition. In *2009 IEEE 12th International Conference on Computer Vision Workshops, ICCV Workshops*, pages 689–696, Sept 2009. 5

[52] B. Zhou, A. Lapedriza, J. Xiao, A. Torralba, and A. Oliva. Learning deep features for scene recognition using places database. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 487–495. Curran Associates, Inc., 2014. 4, 5

[53] Z. Zhu, T. Oskiper, S. Samarasekera, R. Kumar, and H. S. Sawhney. Ten-fold improvement in visual odometry using landmark matching. In *2007 IEEE 11th International Conference on Computer Vision*, pages 1–8, Oct 2007. 6